MERMAID

CHEAT SHEETS

A Complete Reference for Diagram-as-Code

Flowcharts · Sequence Diagrams · Class Diagrams · ER Diagrams · Gantt Charts · Mind Maps · State Diagrams · Git Graph · Themes & Styling · %%{init} Config · CLI & Embeds

12 REFERENCE SHEETS

CONTENTS

Mermaid Reference

Table of Contents

12 reference sheets · syntax · examples · notes

Sheet 01 / 12

Introduction & Syntax Basics

What Mermaid is · how it renders · diagram declaration · comments · live editor

WHAT IS MERMAID?

Mermaid is a JavaScript-based diagramming library that renders text-based markup into SVG diagrams directly in the browser. It lives inside Markdown fenced code blocks (```` ```mermaid ````) and is natively supported by GitHub, GitLab, Obsidian, Notion, VS Code, and many wikis — no image exports, no external tools. The core idea: diagrams as code, version-controlled alongside your documentation.

WHERE MERMAID RENDERS NATIVELY

PLATFORM	HOW
GitHub / GitLab	Fenced ```` ```mermaid ```` block in any Markdown file
Obsidian	Fenced block; built-in, no plugin needed
Notion	/mermaid block or code block set to Mermaid
VS Code	Markdown Preview Mermaid Support extension
Docusaurus	@docusaurus/theme-mermaid plugin
HTML (CDN)	`<script src="mermaid.min.js">` + `mermaid.initialize()`
CLI	`mmdc -i input.mmd -o output.svg` via @mermaid-js/mermaid-cli

DIAGRAM TYPES & DECLARATION KEYWORDS

KEYWORD	DIAGRAM TYPE	SHEET
`flowchart LR`	Flowchart (left→right). Also TD, RL, BT	02
`sequenceDiagram`	Sequence / interaction diagram	03
`classDiagram`	UML class diagram	04
`erDiagram`	Entity-relationship diagram	05
`gantt`	Project timeline / Gantt chart	06
`mindmap`	Mind map (hierarchical)	07
`stateDiagram-v2`	State machine diagram	08
`pie title X`	Pie chart	09
`quadrantChart`	2×2 quadrant chart	09
`timeline`	Linear timeline	10
`gitGraph`	Git branch / commit graph	10

ANNOTATED SYNTAX EXAMPLE

```
%%{init: { "theme": "dark", "logLevel": 1 }}%%
%% This is a comment — ignored by the renderer %%

---
title: My First Diagram
---

flowchart LR
  %% accTitle and accDescr for accessibility
  accTitle: Login Flow
  accDescr: Steps a user takes to log in

  A([Start]) --> B{Has account?}
  B -- Yes --> C[Login Page]
  B -- No  --> D[Register]
  C --> E([End])
```

%%{INIT} CONFIGURATION BLOCK

OPTION	VALUES / NOTES
`theme`	`default` · `dark` · `forest` · `neutral` · `base`
`logLevel`	1=debug … 5=fatal · default: 2
`securityLevel`	`strict` (default) · `loose` · `antiscript` · `sandbox`
`startOnLoad`	`true` / `false` — auto-render on page load
`fontFamily`	Any CSS font string, e.g. `"monospace"`
`flowchart: { curve }`	`linear` · `basis` · `cardinal` · `stepBefore`
`themeVariables`	Deep color overrides — see Sheet 11

LINE-BY-LINE

- → `%%{init: …}%%` — config block; must be the very first line
- → `%% … %%` — comment; stripped before render
- → `--- title: … ---` — YAML front matter for diagram title
- → `flowchart LR` — declares type + direction
- → `accTitle / accDescr` — SVG accessibility attributes
- → `A([…])` — node ID `A` with stadium shape
- → `-->` — default solid arrow connector
- → `-- Yes -->` — arrow with edge label
- → `{…}` — diamond decision node

COMMON MISTAKES

- Putting `%%{init}%%` after the diagram type — it must be the very first line
- Using spaces in node IDs: `My Node` — use camelCase or quotes: `"My Node"`
- Omitting the direction keyword: `flowchart` alone defaults to TD, which surprises new users
- Mixing tabs and spaces in indentation (especially in Gantt and Mindmap)

TIPS & BEST PRACTICES

- Use `mermaid.live` to prototype — it shows parse errors in real time
- Always add `accTitle` and `accDescr` for accessibility in published docs
- Keep node IDs short (`A`, `db1`) — labels go inside the shape brackets
- Use `%%{init: {"theme":"base","themeVariables": {…}}}%%` for full color control

SUMMARY

- Every diagram starts with a type keyword; `%%{init}%%` config goes before it
- Comments use `%% … %%` syntax and are invisible in the render
- Mermaid renders natively in GitHub, Obsidian, Notion, and via CDN or CLI
- Node IDs are separate from node labels — keep IDs short and label-free

Flowcharts

Node shapes · connectors · subgraphs · direction · edge labels · styling

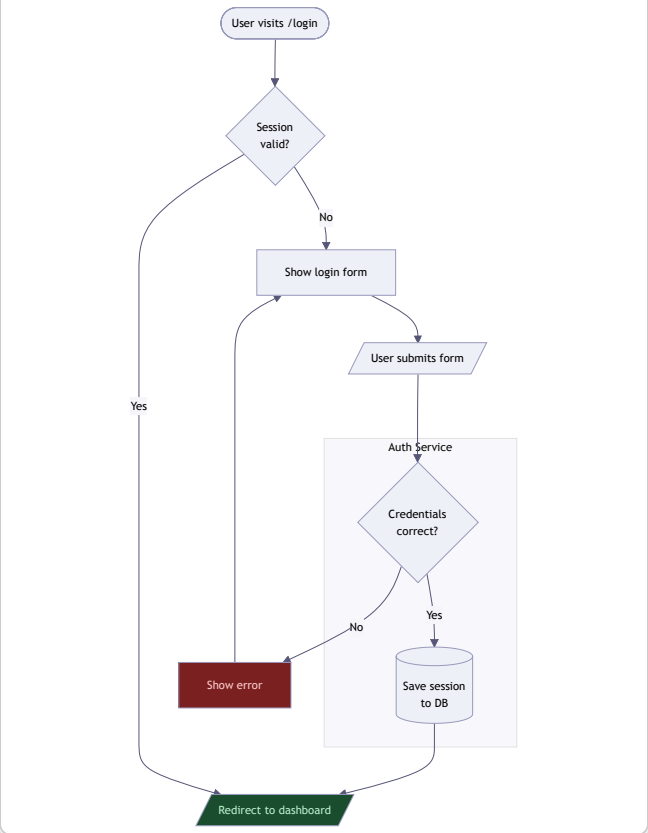
WHAT IS A FLOWCHART?

A flowchart maps a process as a sequence of steps and decisions using nodes (shapes) connected by arrows. In Mermaid, `flowchart` is the most versatile and widely-used diagram type. Direction is set on the first line: TD (top→down), LR (left→right), BT (bottom→top), RL (right→left). Typical uses: software logic, user journeys, decision trees, CI/CD pipelines, onboarding flows.

USE CASES

- Software / algorithm logic
- User journey & UX flows
- Business process documentation
- Decision trees & if/else maps
- CI/CD pipeline visualization
- Troubleshooting / runbooks

RENDERED OUTPUT



CODE EXAMPLE — USER AUTHENTICATION FLOW

```
%%{init: {"theme": "dark"}}%%
flowchart TD
  A([User visits /login]) --> B{Session\nvalid?}
  B -- Yes --> C[/Redirect to dashboard/]
  B -- No  --> D[Show login form]
  D --> E[/User submits form/]
  E --> F{Credentials\ncorrect?}
  F -- Yes --> G[(Save session\nto DB)]
  F -- No  --> H[Show error]
  G --> C
  H --> D
  subgraph auth [Auth Service]
    F
    G
  end
  style H fill:#7b2020,color:#ffcccc
  style C fill:#1a4d2e,color:#b9f0cb
```

NODE SHAPES

SYNTAX	SHAPE
`id[Text]`	Rectangle — default process step
`id(Text)`	Rounded rectangle
`id([Text])`	Stadium / pill — start or end
`id{Text}`	Diamond — decision / branch
`id{{Text}}`	Hexagon
`id[/Text/]`	Parallelogram — input / output
`id[\Text\]`	Parallelogram (alt direction)
`id[/Text\]`	Trapezoid
`id[\Text/]`	Trapezoid (inverted)
`id[(Text)]`	Cylinder — database
`id((Text))`	Circle
`id(((Text)))`	Double circle
`id>Text]`	Asymmetric / flag shape

CONNECTORS & ARROWS

SYNTAX	MEANING
`A --> B`	Solid arrow (open head)
`A --- B`	Line — no arrowhead
`A -- label --> B`	Arrow with edge label
`A -->|label| B`	Arrow with label (alt syntax)
`A ==> B`	Thick arrow
`A == label ==> B`	Thick arrow with label
`A -.-> B`	Dotted arrow
`A -. label .-> B`	Dotted arrow with label
`A --o B`	Circle end
`A --x B`	Cross / terminated end
`A <--> B`	Bidirectional arrow
`A o--o B`	Circle on both ends
`A x--x B`	Cross on both ends

SUBGRAPHS & STYLING

SYNTAX	PURPOSE
`subgraph id [Title]`	Open a named subgraph group
`end`	Close subgraph block
`direction LR`	Override direction inside a subgraph
`style id fill:,color:`	Inline style a single node
`classDef name fill:,stroke:`	Define a reusable CSS class
`class A,B name`	Apply a classDef to listed nodes
`A:::name --> B`	Apply class inline on a node
`linkStyle 0 stroke:#f00`	Style an edge by 0-based index
`\n in label`	Line break inside a node label
`"quoted id"`	Node ID containing spaces

COMMON MISTAKES

- Using `&`, `<`, or `>` in labels without quoting — wrap label in `"…"`
- Forgetting `end` to close a subgraph — diagram silently fails to render
- Re-using the same node ID with different labels — IDs are global; reuse merges nodes
- `linkStyle` index is 0-based and counts every edge in declaration order

TIPS & BEST PRACTICES

- Prefer `LR` for process flows; use `TD` for hierarchies and trees
- Use `classDef` + `class` instead of repeating `style` on every node
- Group related nodes in subgraphs — makes large diagrams scannable
- Use stadium `([…])` for start/end; reserve diamond `{…}` for decisions only

SUMMARY

- Declare with `flowchart TD|LR|BT|RL` — direction keyword is required
- 13 node shapes; shape is set by bracket type, not a keyword
- 13 connector styles: solid, dotted, thick, labeled, bidirectional
- Subgraphs group nodes visually; `classDef` enables reusable styling

Sequence Diagrams

Participants · actors · message arrows · loops · alt/opt · activation bars

WHAT IS A SEQUENCE DIAGRAM?

A sequence diagram shows how objects or systems interact over time, with each participant as a vertical lifeline and messages as horizontal arrows between them. It captures the *order* of operations — who calls whom, and when. Typical uses: API request/response flows, authentication handshakes, microservice communication, event-driven systems, and user-to-system interactions.

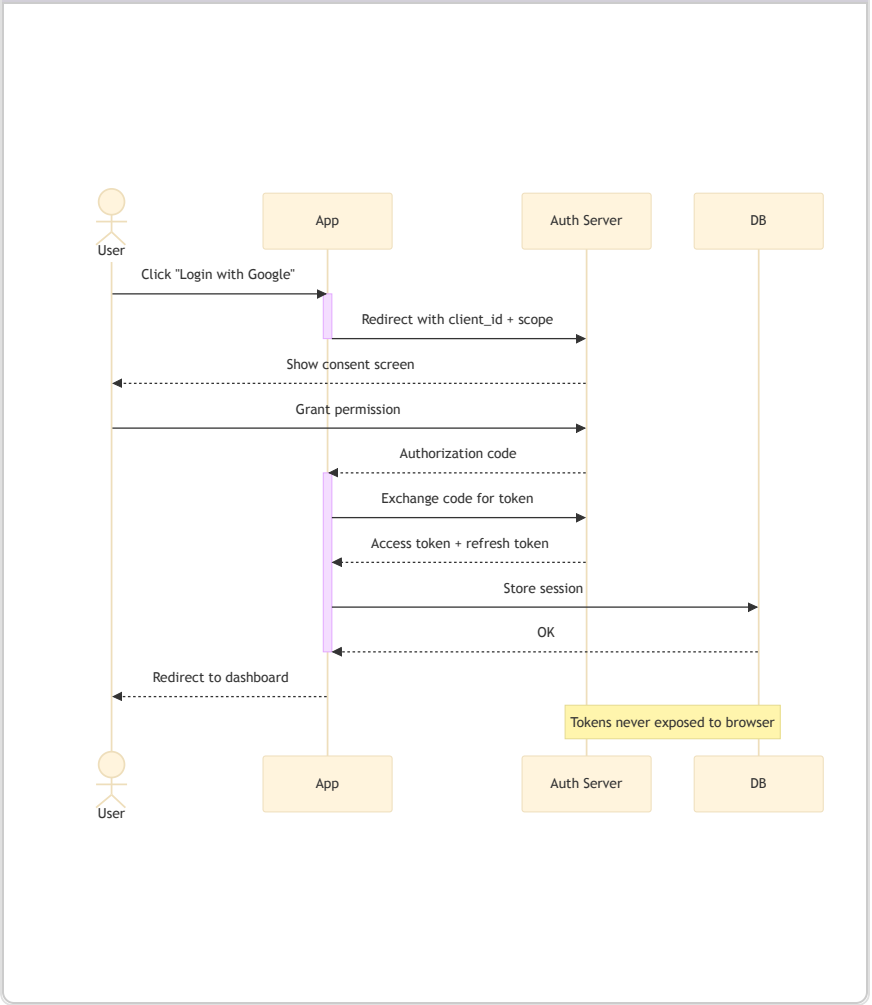
USE CASES

- API & HTTP request/response flows
- Auth handshakes (OAuth, JWT)
- Microservice call chains
- Event-driven / pub-sub systems
- User ↔ system interaction flows
- Database query sequences

CODE EXAMPLE — OAUTH LOGIN FLOW

```
sequenceDiagram
  accTitle: OAuth Login Flow
  accDescr: Shows how a user authenticates via OAuth provider

  actor U as User
  participant App
  participant Auth as Auth Server
  participant DB

  U->>App: Click "Login with Google"
  activate App
  App->>Auth: Redirect with client_id + scope
  deactivate App

  Auth-->>U: Show consent screen
  U->>Auth: Grant permission

  Auth-->>App: Authorization code
  activate App
  App->>Auth: Exchange code for token
  Auth-->>App: Access token + refresh token

  App->>DB: Store session
  DB-->>App: OK
  deactivate App
  App-->>U: Redirect to dashboard

  Note over Auth,DB: Tokens are never exposed to the browser
```

RENDERED OUTPUT



PARTICIPANTS & ACTORS

SYNTAX	MEANING
`participant A`	Rectangle lifeline box
`participant A as Label`	Lifeline with display alias
`actor A`	Stick-figure actor (human user)
`actor A as Label`	Actor with display alias
`create participant A`	Lifeline created mid-diagram
`destroy A`	Terminate lifeline with X
`box Title`	Group participants in a box
`end`	Close a box group

MESSAGE ARROW TYPES

SYNTAX	MEANING
`A->>B: msg`	Solid line, open arrowhead (sync call)
`A-->>B: msg`	Dashed line, open arrowhead (response)
`A->B: msg`	Solid line, no arrowhead
`A-->B: msg`	Dashed line, no arrowhead
`A-xB: msg`	Solid line, X end (async / lost)
`A--xB: msg`	Dashed line, X end
`A-)B: msg`	Solid line, open async arrow
`A--)B: msg`	Dashed line, open async arrow
`activate A`	Show activation bar on lifeline
`deactivate A`	End activation bar
`A->>+B: msg`	Send + auto-activate B
`B-->>-A: msg`	Reply + auto-deactivate B

CONTROL FLOW BLOCKS

SYNTAX	PURPOSE
`loop Label`	Repeating block with label
`alt Label`	Alternative paths (if/else)
`else Label`	Alternative branch inside alt
`opt Label`	Optional block (executes or not)
`par Label`	Parallel execution block
`and Label`	Additional parallel branch
`critical Label`	Critical region block
`break Label`	Break out of a sequence
`end`	Close any block
`Note right of A: text`	Annotation on right of lifeline
`Note left of A: text`	Annotation on left of lifeline
`Note over A,B: text`	Annotation spanning A to B

COMMON MISTAKES

- Using `->` instead of `->>` — single-arrow has no arrowhead, which looks broken
- Mismatched `activate` / `deactivate` calls cause bars to stack or never close
- Forgetting `end` to close `loop`, `alt`, or `par` blocks
- Declaring a participant mid-diagram without `create participant` — it renders but lifeline starts at top

TIPS & BEST PRACTICES

- Declare all participants at the top to control their left-to-right order
- Use `+/-` shorthand on arrows instead of separate `activate/deactivate` lines
- Use `actor` only for humans; use `participant` for all services and systems
- Keep diagrams to 5–7 participants — beyond that, split into multiple diagrams

SUMMARY

- `participant` = box lifeline · `actor` = stick figure
- 8 arrow types: solid/dashed × open/none/X/async
- Control blocks: `loop` · `alt/else` · `opt` · `par/and` · `break`
- `Note over A,B` spans multiple lifelines; `activate/deactivate` show call depth

Class Diagrams

Classes · members · visibility · relationships · inheritance · interfaces

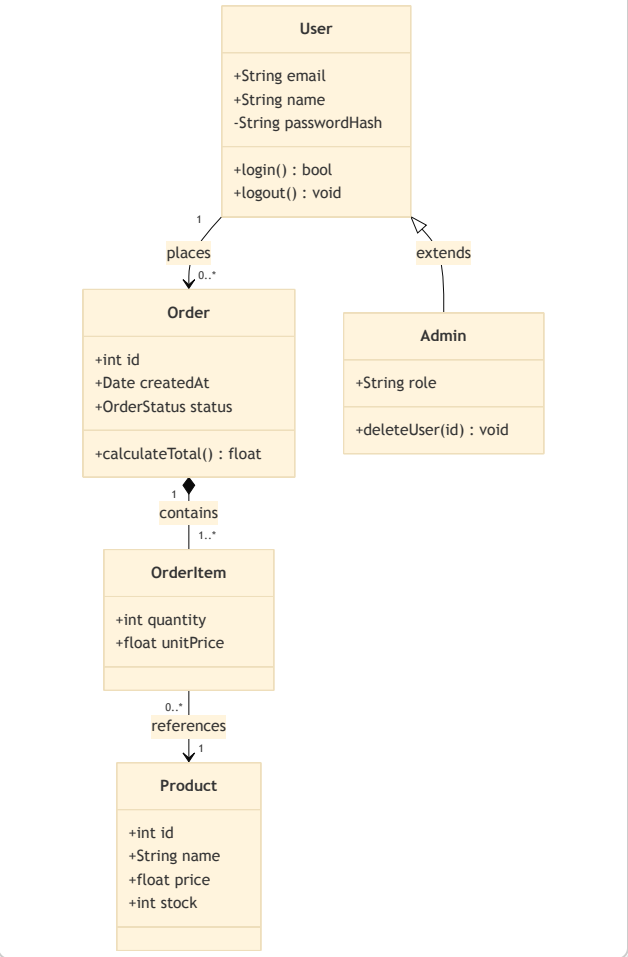
WHAT IS A CLASS DIAGRAM?

A class diagram is the cornerstone of UML object-oriented design. It shows classes, their attributes and methods, and the relationships between them — inheritance, composition, aggregation, association, and dependency. Use it to document a system's structure before building it, or to reverse-engineer existing code. Typical uses: OOP architecture documentation, domain modeling, API design, database schema planning.

USE CASES

- OOP architecture & design docs
- Domain model visualization
- API / SDK structure documentation
- Database schema planning
- Reverse-engineering existing code
- Interface & abstract class planning

RENDERED OUTPUT



CODE EXAMPLE — E-COMMERCE DOMAIN

```
classDiagram
  class User {
    +String email
    +String name
    -String passwordHash
    +login() bool
    +logout() void
  }

  class Order {
    +int id
    +Date createdAt
    +OrderStatus status
    +calculateTotal() float
  }

  class Product {
    +int id
    +String name
    +float price
    +int stock
  }

  class OrderItem {
    +int quantity
    +float unitPrice
  }

  class Admin {
    +String role
    +deleteUser(id) void
  }

  User <|-- Admin : extends
  User "1" --> "0..*" Order : places
  Order "1" *-- "1..*" OrderItem : contains
  OrderItem "0..*" --> "1" Product : references
```

MEMBER SYNTAX & VISIBILITY

SYNTAX	MEANING
`+name : Type`	Public attribute
`-name : Type`	Private attribute
`#name : Type`	Protected attribute
`~name : Type`	Package / internal attribute
`+method() RetType`	Public method
`method()*`	Abstract method (italic)
`method()$`	Static method (underlined)
`classname()`	Constructor convention
`<<interface>>`	Interface annotation
`<<abstract>>`	Abstract class annotation
`<<enumeration>>`	Enum annotation

RELATIONSHIP TYPES

SYNTAX	RELATIONSHIP
`A <|-- B`	Inheritance — B extends A
`A <|.. B`	Realization — B implements A
`A *-- B`	Composition — B owned by A
`A o-- B`	Aggregation — B part of A
`A --> B`	Association — A uses B
`A -- B`	Link (plain line)
`A ..> B`	Dependency — A depends on B
`A .. B`	Dashed link
`A "1" --> "0..*" B`	Labeled cardinality on either end
`A --> B : label`	Relationship label after colon

NAMESPACES & STYLING

SYNTAX	PURPOSE
`namespace Domain {}`	Group classes in a named namespace
`class A:::cssClass`	Apply CSS class to a node
`classDef name fill:,stroke:`	Define a reusable style class
`direction TB`	Set layout direction (TB, LR, etc.)
`note for A "text"`	Attach a note to a class

COMMON MISTAKES

- Confusing composition `*--` (owned, destroyed together) with aggregation `o--` (shared, independent)
- Arrow direction matters: `A <|-- B` means B inherits A — reversed from spoken "A extends B"
- Forgetting `<<interface>>` annotation — plain classes and interfaces look identical without it
- Using spaces in class names — use PascalCase; spaces break parsing

TIPS & BEST PRACTICES

- Show only the most important attributes — omit getters, setters, and boilerplate
- Use cardinality labels (`"1"` , `"0..*"`) on every association to clarify intent
- Use `namespace` to group related classes — prevents diagram sprawl
- Combine with a sequence diagram: class diagram for structure, sequence for behavior

SUMMARY

- Visibility: `+` public · `-` private · `#` protected · `~` package
- Modifiers: `*` abstract · `$` static · `<<interface>>` annotation
- 8 relationship types from inheritance to dependency
- Cardinality labels on both ends of any association arrow

Entity Relationship Diagrams

Entities · attributes · cardinality · relationships · keys · data modeling

WHAT IS AN ER DIAGRAM?

An entity-relationship diagram models the data structure of a system — the things that exist (entities), what you know about them (attributes), and how they relate to each other (relationships with cardinality). Mermaid uses Crow's Foot notation for cardinality. Typical uses: relational database schema design, data dictionary documentation, API data model planning, and communicating data structure to non-technical stakeholders.

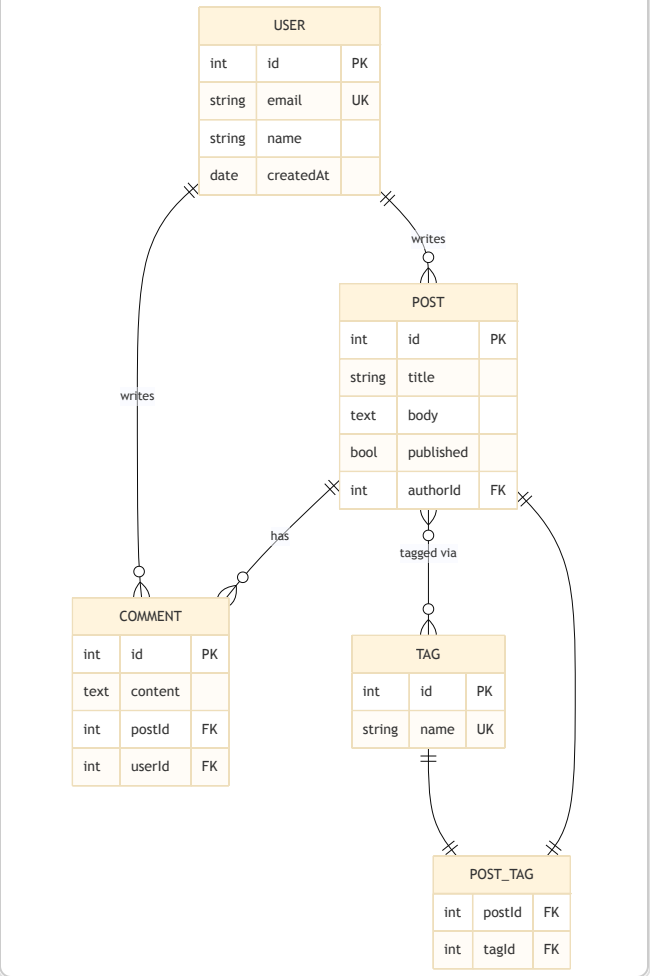
USE CASES

- Relational DB schema design
- Data dictionary documentation
- API payload / data model docs
- ORM model planning
- Stakeholder data communication
- Legacy system reverse engineering

CODE EXAMPLE — BLOG PLATFORM SCHEMA

```
erDiagram
  USER {
    int     id          PK
    string  email       UK
    string  name
    date    createdAt
  }

  POST {
    int     id          PK
    string  title
    text    body
    bool    published
    int     authorId    FK
  }

  COMMENT {
    int     id          PK
    text    content
    int     postId      FK
    int     userId      FK
  }

  TAG {
    int     id          PK
    string  name        UK
  }

  POST_TAG {
    int     postId      FK
    int     tagId       FK
  }

  USER ||--o{ POST    : "writes"
  POST ||--o{ COMMENT : "has"
  USER ||--o{ COMMENT : "writes"
  POST }o--o{ TAG     : "tagged via"
  POST ||--|| POST_TAG : ""
  TAG  ||--|| POST_TAG : ""
```

RENDERED OUTPUT



CARDINALITY (CROW'S FOOT)

SYNTAX	LEFT END	RIGHT END
`||--||`	Exactly one	Exactly one
`||--o|`	Exactly one	Zero or one
`||--o{`	Exactly one	Zero or many
`||--|{`	Exactly one	One or many
`o|--||`	Zero or one	Exactly one
`o{--||`	Zero or many	Exactly one
`}o--o{`	Zero or many	Zero or many
`}|--|{`	One or many	One or many

ATTRIBUTE TYPES & KEYS

SYNTAX	MEANING
`int id PK`	Primary key — shown with key icon
`string email UK`	Unique key constraint
`int userId FK`	Foreign key reference
`type name "comment"`	Attribute with inline comment string
`string`	Common types: `int` · `string` · `float`
`bool / date / text`	Additional common attribute types

RELATIONSHIP SYNTAX

COMPONENT	OPTIONS
`Left crow marker`	| one · o zero-or-one · } many
`Line style`	-- identifying · `..` non-identifying
`Right crow marker`	| one · o zero-or-one · { many
`Label`	`ENTITY ||--o{ OTHER : "label"` — label in quotes
`No label`	Use empty string `""` — label is required by parser
`Identifying`	Child cannot exist without parent (solid line --)
`Non-identifying`	Child can exist independently (dashed line `..`)

COMMON MISTAKES

- Omitting the relationship label — it is required; use `""` if you want none visible
- Confusing identifying (`--`) vs non-identifying (`..`) lines — child existence rules differ
- Using lowercase entity names — convention is UPPER_SNAKE_CASE for entities
- Mixing cardinality direction — left side describes the entity on the left, right side the right entity

TIPS & BEST PRACTICES

- Always mark `PK` , `FK` , and `UK` — it communicates intent clearly to reviewers
- Model junction tables (e.g. `POST_TAG`) explicitly rather than using many-to-many directly
- Keep attribute types consistent with your actual DB types — don't use `string` if your DB uses `VARCHAR`
- Use relationship labels as verb phrases: `"writes"` , `"belongs to"` , `"contains"`

SUMMARY

- Entities declared as blocks; attributes listed as `type name [PK|FK|UK]`
- Crow's Foot cardinality: `|` one · `o` zero-or-one · `{` / `}` many
- Solid line `--` = identifying; dashed `..` = non-identifying relationship
- All relationships require a label — use `""` for an invisible one

Gantt Charts

Tasks · sections · dates · milestones · dependencies · status modifiers

WHAT IS A GANTT CHART?

A Gantt chart displays a project schedule as a horizontal bar chart — each task is a bar spanning its start-to-end duration along a time axis. Sections group related tasks, and tasks can depend on each other. Mermaid supports absolute dates, relative durations, milestones, and status modifiers (`done`, `active`, `crit`). Typical uses: sprint planning, project roadmaps, release schedules, and milestone tracking.

USE CASES

- Sprint & iteration planning
- Project roadmaps & timelines
- Release & deployment schedules
- Milestone tracking
- Resource allocation overview
- Dependency visualization

CODE EXAMPLE — PRODUCT LAUNCH SPRINT

```
gantt
  title Product Launch — Q3 Sprint
  dateFormat  YYYY-MM-DD
  axisFormat  %b %d

  section Discovery
    Stakeholder interviews  : done,    disc1, 2024-07-01, 5d
    Requirements doc        : done,    disc2, after disc1, 4d
    Design review           : active,  disc3, after disc2, 3d

  section Development
    API endpoints           : crit,    dev1,  after disc3, 7d
    Frontend components     :          dev2,  after disc3, 10d
    Integration             : crit,    dev3,  after dev1,  4d

  section QA
    Unit tests              :          qa1,   after dev2,  3d
    UAT                     : crit,    qa2,   after dev3,  4d

  section Release
    Staging deploy          :          rel1,  after qa2,   1d
    Go-live                 : milestone, rel2, after rel1, 0d
```

RENDERED OUTPUT

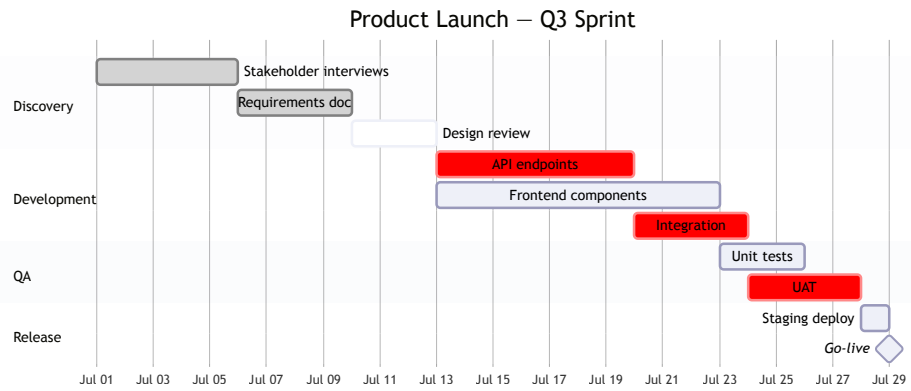


CHART-LEVEL CONFIGURATION

DIRECTIVE	PURPOSE / VALUES
`title Text`	Chart heading displayed above the bars
`dateFormat fmt`	Input date format, e.g. `YYYY-MM-DD` · `DD/MM/YYYY`
`axisFormat fmt`	Axis label format: `%Y` · `%b` · `%d` · `%H:%M`
`tickInterval 1week`	Axis tick spacing: `1day` · `1week` · `1month`
`weekday monday`	First day of week for weekly ticks
`excludes weekends`	Skip weekends in duration calculations
`todayMarker on/off`	Show/hide the current date marker line
`section Label`	Group tasks under a labeled swimlane

TASK DECLARATION SYNTAX

FORMAT	MEANING
`Label : id, start, dur`	Task with ID, absolute start, duration
`Label : id, after X, dur`	Task starts after task with ID X
`Label : start, dur`	Task without explicit ID
`dur: 5d`	Duration in days (d), hours (h), weeks (w)
`start: YYYY-MM-DD`	Absolute start date (must match dateFormat)
`after X, Y`	Start after the later of tasks X and Y

STATUS MODIFIERS

MODIFIER	EFFECT
`done`	Task rendered as completed (grey fill)
`active`	Task highlighted as in progress
`crit`	Critical path — rendered in red
`milestone`	Renders as a diamond point, not a bar
`crit, done`	Modifiers can be combined with a comma
`crit, active`	Critical and currently active task

COMMON MISTAKES

- Date format mismatch — `dateFormat` must match every date string in the chart exactly
- Using `after X` with a task ID that hasn't been declared yet — IDs must be declared before referenced
- Setting duration `0d` on a regular task — use `milestone` modifier instead
- Forgetting `axisFormat` — the default shows Unix timestamps and is hard to read

TIPS & BEST PRACTICES

- Use `after X` dependencies instead of hardcoded dates — chart stays valid when dates shift
- Use `crit` to mark your critical path — makes bottlenecks immediately visible
- Add `excludes weekends` for real project timelines to get accurate durations
- Keep section names short — they appear as labels on the left and space is limited

SUMMARY

- `dateFormat` sets input format; `axisFormat` sets display format — both required
- Task syntax: `Label : [modifiers,] [id,] start, duration`
- Status modifiers: `done` · `active` · `crit` · `milestone`
- `after id` chains tasks; `section` groups them into swimlanes

Mind Maps

Hierarchy · node shapes · indentation · icons · classes · brainstorming

WHAT IS A MIND MAP?

A mind map is a radial hierarchy diagram that branches outward from a central concept. In Mermaid, structure is defined entirely by **indentation** — each level of indentation is one level deeper in the tree. There are no explicit connection declarations; parent-child relationships are implied by whitespace. Typical uses: brainstorming, knowledge organization, content outlines, decision exploration, and study notes.

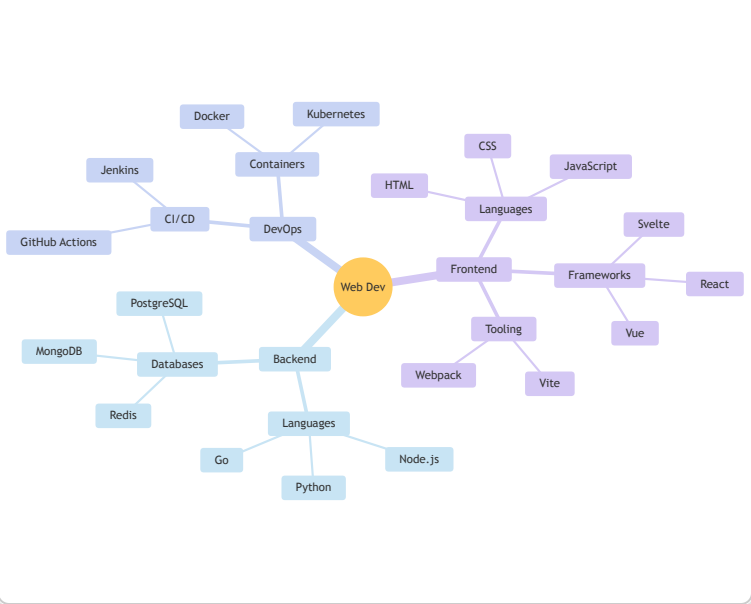
USE CASES

- Brainstorming & idea capture
- Content outlines & structures
- Project scope exploration
- Study notes & concept maps
- Decision tree exploration
- Tech stack & system overview

CODE EXAMPLE — FULL-STACK WEB DEV SKILLS

```
mindmap
  root((Web Dev))
    Frontend
      ::icon(fa fa-desktop)
      Languages
        HTML
        CSS
        JavaScript
      Frameworks
        React
        Vue
        Svelte
      Tooling
        Vite
        Webpack
    Backend
      ::icon(fa fa-server)
      Languages
        Node.js
        Python
        Go
      Databases
        PostgreSQL
        MongoDB
        Redis
    DevOps
      ::icon(fa fa-cloud)
      CI/CD
        GitHub Actions
        Jenkins
      Containers
        Docker
        Kubernetes
```

RENDERED OUTPUT



NODE SHAPES

SYNTAX	SHAPE
`id((Text))`	Circle — typically used for root
`id(Text)`	Rounded rectangle
`id[Text]`	Rectangle / square
`id)Text(`	Cloud / explosion shape
`id))Text((`	Bang / starburst shape
`id{{Text}}`	Hexagon
`Text`	Default shape (no brackets = rounded rect)

ICONS & CLASSES

SYNTAX	PURPOSE
`::icon(fa fa-name)`	Font Awesome icon on the node below
`::icon(mdi-name)`	Material Design icon (if loaded)
`:::className`	Apply a CSS class to the node
`classDef name fill:,stroke:`	Define reusable style (in %%{init}%% or global)

HIERARCHY & SYNTAX RULES

RULE	DETAIL
`Indentation = depth`	Each extra indent level = one child level deeper
`Use spaces, not tabs`	Tabs cause parse errors — use consistent spaces
`First node = root`	The least-indented node after `mindmap` is the root
`One root only`	Mermaid mind maps support exactly one root node
`No connections`	Relationships are implicit — no arrow syntax used
`::icon goes under node`	Icon line must be indented under the node it decorates
`No back-references`	Nodes cannot reference earlier nodes (tree only, no cycles)

COMMON MISTAKES

- Using tabs for indentation — mind maps require consistent spaces only
- Declaring multiple root-level nodes — only the first is treated as root; others become orphans
- Placing `::icon()` at the same indent as its node instead of one level deeper
- Trying to create cross-links between branches — mind maps are strictly trees

TIPS & BEST PRACTICES

- Use `((root))` circle shape for the central concept — it visually anchors the diagram
- Keep branch depth to 3–4 levels — deeper trees become unreadable
- Use icons (`::icon`) on second-level branches to make categories scannable
- Mind maps pair well with a flowchart — use the mind map to explore, then flowchart to formalize

SUMMARY

- Structure is purely indentation-based — no arrow syntax
- 7 node shapes; default (no brackets) renders as rounded rectangle
- `::icon()` adds Font Awesome or Material icons to any node
- One root node required; tabs break parsing — use spaces only

State Diagrams

States · transitions · composite states · concurrency · notes · forks

WHAT IS A STATE DIAGRAM?

A state diagram (UML statechart) models the lifecycle of an object — the distinct states it can be in, and the transitions triggered by events or conditions that move it between those states. Mermaid uses `stateDiagram-v2` for the modern syntax. Typical uses: order lifecycle management, UI component state machines, authentication flows, IoT device states, and game logic.

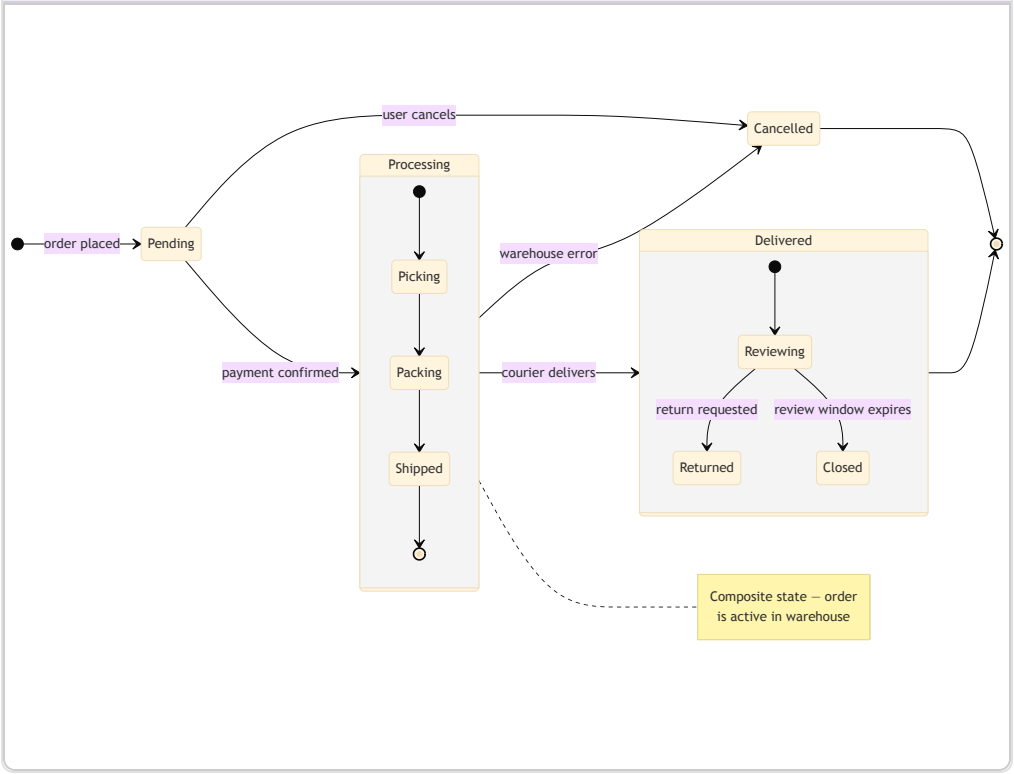
USE CASES

- Order & fulfillment lifecycle
- UI component state machines
- Auth session states
- IoT device state tracking
- Game logic & NPC behavior
- Workflow & approval pipelines

CODE EXAMPLE — ORDER LIFECYCLE

```
stateDiagram-v2
  accTitle: Order Lifecycle
  direction LR

  [*] --> Pending : order placed

  Pending --> Processing : payment confirmed
  Pending --> Cancelled  : user cancels

  state Processing {
    [*] --> Picking
    Picking --> Packing
    Packing --> Shipped
    Shipped --> [*]
  }

  Processing --> Cancelled : warehouse error
  Processing --> Delivered : courier delivers

  state Delivered {
    [*] --> Reviewing
    Reviewing --> Returned : return requested
    Reviewing --> Closed   : review window expires
  }

  Cancelled --> [*]
  Delivered --> [*]

  note right of Processing
    Composite state — order
    is active in warehouse
  end note
```

RENDERED OUTPUT



STATE & TRANSITION SYNTAX

SYNTAX	MEANING
`[*] --> A`	Initial pseudostate → state A
`A --> [*]`	State A → final pseudostate
`A --> B`	Transition from A to B (no label)
`A --> B : event`	Labeled transition triggered by event
`state "Long Name" as S`	State with display alias
`state A { … }`	Composite / nested state block
`direction LR / TD`	Layout direction for the whole diagram

CONCURRENCY, FORKS & NOTES

SYNTAX	PURPOSE
`state fork_id <<fork>>`	Fork pseudostate (splits into parallel)
`state join_id <<join>>`	Join pseudostate (merges parallel paths)
`state choice_id <<choice>>`	Choice / decision pseudostate
`--`	Concurrent region separator (inside composite)
`note right of S`	Annotation on right side of state S
`note left of S`	Annotation on left side of state S
`end note`	Close a multi-line note block
`classDef name fill:,color:`	Define reusable node style
`class A,B name`	Apply classDef to states

COMMON MISTAKES

- Using `stateDiagram` instead of `stateDiagram-v2` — v1 has limited syntax and fewer features
- Forgetting `[*]` entry/exit in composite states — nested states need their own initial pseudostate
- No transition to `[*]` from terminal states — the diagram looks like it has no end
- Forgetting `end note` to close a multi-line note block

TIPS & BEST PRACTICES

- Use `direction LR` for lifecycle flows — left-to-right reads like a timeline
- Label every transition with the event or guard that triggers it
- Use composite states to hide sub-detail — viewers can see the big picture first
- Keep state names as noun phrases: `Pending`, `Processing`, not verbs like `Process`

SUMMARY

- Use `stateDiagram-v2` — always prefer v2 syntax
- `[*]` is the initial and final pseudostate; every diagram needs both
- Composite states (`state A { … }`) nest sub-machines; `--` adds concurrency
- Pseudostates: `<<fork>>` · `<<join>>` · `<<choice>>`

Pie & Quadrant Charts

Pie: proportions · showData · title · Quadrant: axes · points · labeling

PIE CHARTS

Pie charts show proportional composition of a whole. Mermaid's `pie` diagram is intentionally simple — a title, optional `showData` flag to display raw values, and a flat list of labeled slices with numeric values. Values are automatically normalized to percentages. Use for: survey results, budget breakdowns, market share, traffic source splits, and test coverage ratios.

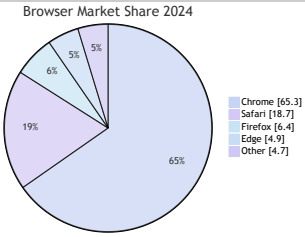
QUADRANT CHARTS

A quadrant chart plots items on a 2×2 grid defined by two axes, dividing the space into four labeled quadrants. Each point has an x/y coordinate between 0 and 1. Use for: prioritization matrices (effort vs. impact), product positioning, risk assessment, competitive analysis, and the classic "urgent vs. important" framework.

PIE CHART — CODE EXAMPLE

```
pie showData
  title Browser Market Share 2024
  "Chrome"  : 65.3
  "Safari"  : 18.7
  "Firefox" : 6.4
  "Edge"    : 4.9
  "Other"   : 4.7
```

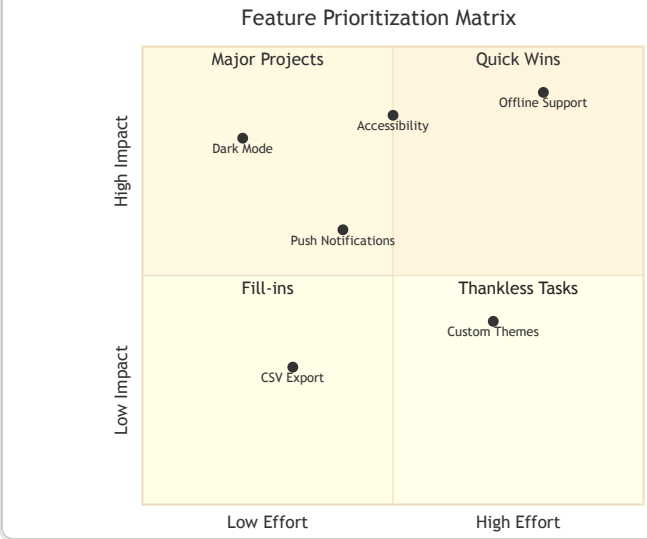
PIE — RENDERED OUTPUT



QUADRANT CHART — CODE EXAMPLE

```
quadrantChart
  title Feature Prioritization Matrix
  x-axis Low Effort --> High Effort
  y-axis Low Impact --> High Impact

  quadrant-1 Quick Wins
  quadrant-2 Major Projects
  quadrant-3 Fill-ins
  quadrant-4 Thankless Tasks

  Dark Mode:          [0.2, 0.8]
  Offline Support:    [0.8, 0.9]
  Push Notifications: [0.4, 0.6]
  CSV Export:         [0.3, 0.3]
  Custom Themes:      [0.7, 0.4]
  Accessibility:      [0.5, 0.85]
```

QUADRANT — RENDERED OUTPUT



PIE CHART SYNTAX

DIRECTIVE	PURPOSE
`pie`	Declare a pie chart
`pie showData`	Show raw values in the legend
`title Text`	Chart heading above the pie
`"Label" : value`	Slice with label and numeric value
`%%{init: {pie: {textPosition: 0.6}}}%%`	Move label text radial position (0–1)

QUADRANT CHART SYNTAX

DIRECTIVE	PURPOSE	DIRECTIVE	PURPOSE
`quadrantChart`	Declare a quadrant chart	`quadrant-2 Label`	Top-left quadrant label
`title Text`	Chart heading	`quadrant-3 Label`	Bottom-left quadrant label
`x-axis Low --> High`	X axis label: left end → right end	`quadrant-4 Label`	Bottom-right quadrant label
`y-axis Low --> High`	Y axis label: bottom → top	`Point Name: [x, y]`	Plot a point at coordinates 0–1
`quadrant-1 Label`	Top-right quadrant label		

COMMON MISTAKES

- Pie slice values don't need to sum to 100 — Mermaid normalizes automatically; summing to 100 is fine but not required
- Quadrant point coordinates must be between 0 and 1 — values outside this range are clipped or cause errors
- Quadrant labels 1–4 map to specific quadrants (1=top-right) — easy to mix up
- Missing quotes around pie slice labels causes parse errors

TIPS & BEST PRACTICES

- Use `showData` on pie charts when the actual numbers matter to the audience
- Limit pie charts to 5–7 slices — more makes the legend unreadable
- In quadrant charts, use evenly-spread points — clustering everything in one quadrant defeats the purpose
- Use quadrant charts for prioritization workshops — they're easy for non-technical stakeholders to read

SUMMARY

- Pie: `pie [showData]` → `title` → `"Label" : value` slices
- Quadrant: `x-axis` / `y-axis` labels → 4 quadrant names → `Point: [x, y]`
- Pie values auto-normalize; quadrant coordinates are 0–1 scale
- Both charts are best for quick visual communication rather than precise data analysis

Timeline & Git Graph

Timeline: sections · events · Git Graph: commit · branch · merge · cherry-pick

TIMELINE CHARTS

A timeline displays events or milestones on a horizontal time axis, grouped into labeled sections. Unlike Gantt, it has no duration bars — just points or short labels in chronological order. Typical uses: product history, company milestones, project phase summaries, historical overviews, and roadmap communications.

GIT GRAPH

Git Graph visualizes a repository's commit and branch history — branches, commits, merges, and cherry-picks. It mirrors what you'd see in `git log --graph`. Typical uses: documenting branching strategies, PR/merge workflows, explaining Git flow or trunk-based development, and teaching Git concepts.

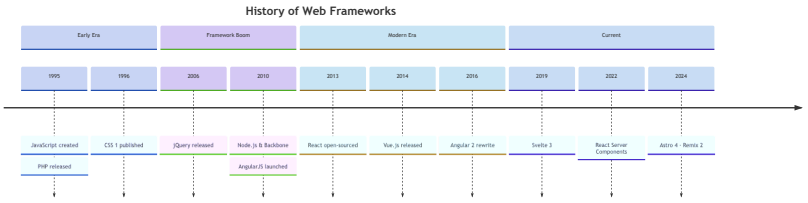
TIMELINE — CODE EXAMPLE

```
timeline
  title History of Web Frameworks
  section Early Era
    1995 : JavaScript created
         : PHP released
    1996 : CSS 1 published
  section Framework Boom
    2006 : jQuery released
    2010 : Node.js & Backbone
         : AngularJS launched
  section Modern Era
    2013 : React open-sourced
    2014 : Vue.js released
    2016 : Angular 2 rewrite
  section Current
    2019 : Svelte 3
    2022 : React Server Components
    2024 : Astro 4 · Remix 2
```

GIT GRAPH — CODE EXAMPLE

```
%%{init: { "gitGraph": {"rotateCommitLabel": false} }}%%
gitGraph LR:
  commit id: "init"
  commit id: "add readme"

  branch develop
  checkout develop
  commit id: "feature scaffold"
  commit id: "add tests"

  branch feature/auth
  checkout feature/auth
  commit id: "add login"
  commit id: "add JWT"

  checkout develop
  merge feature/auth id: "merge auth"

  branch release/1.0
  checkout release/1.0
  commit id: "bump version" tag: "v1.0.0"

  checkout main
  merge release/1.0

  checkout develop
  cherry-pick id: "bump version"
  commit id: "hotfix typo"
```

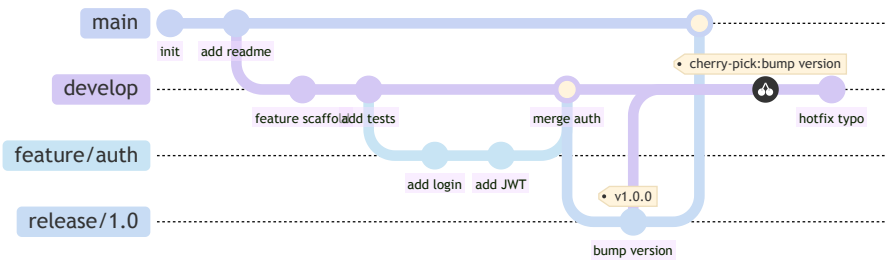
TIMELINE — RENDERED



TIMELINE SYNTAX

SYNTAX	PURPOSE
`timeline`	Declare a timeline diagram
`title Text`	Heading above the timeline
`section Label`	Group events into a labeled band
`Period : Event`	Single event under a time period
`: Event`	Additional event on same period (indent)

GIT GRAPH — RENDERED



GIT GRAPH COMMANDS

SYNTAX	MEANING
`commit`	Add a commit to current branch
`commit id: "msg"`	Commit with custom label
`commit tag: "v1.0"`	Commit with a version tag
`commit type: HIGHLIGHT`	Types: `NORMAL` · `REVERSE` · `HIGHLIGHT`
`branch name`	Create and stay on current branch
`checkout name`	Switch to named branch
`merge name`	Merge named branch into current
`merge name id: "msg"`	Merge with custom commit label
`cherry-pick id: "X"`	Cherry-pick commit with given id
`gitGraph LR:`	Left-to-right layout (default: LR)
`gitGraph TB:`	Top-to-bottom layout

COMMON MISTAKES

- Git: `cherry-pick` requires the source commit to have an explicit `id` — anonymous commits can't be cherry-picked
- Git: merging a branch that hasn't diverged renders oddly — add at least one commit after branching
- Timeline: periods with no events still need a colon — omitting it causes parse errors
- Git: `checkout` before `branch` will throw an error — always create the branch first

TIPS & BEST PRACTICES

- Git: give every commit an `id` — unlabeled commits are hard to reference and read
- Git: use `%%{init: {"gitGraph": {"rotateCommitLabel":false}}}%%` to prevent label rotation on LR layouts
- Timeline: use `section` groupings to create visual bands — helps readers parse time periods quickly
- Keep Git graphs to the branch strategy being communicated — don't show every commit

SUMMARY

- Timeline: `section` groups → `Period : Event` entries; multi-event periods use extra `: Event` lines
- Git Graph: `commit` → `branch` → `checkout` → `merge` mirrors real Git commands
- Git commit types: `NORMAL` · `REVERSE` · `HIGHLIGHT`; tags added with `tag:`
- `cherry-pick id: "X"` requires an explicit commit id to reference

Themes & Styling

Built-in themes · classDef · style · linkStyle · themeVariables · CSS overrides

HOW MERMAID STYLING WORKS

Mermaid has four layers of styling, from broadest to most specific: **(1)** built-in `theme` sets global palette; **(2)** `themeVariables` overrides individual palette tokens; **(3)** `classDef` / `class` applies reusable CSS rules to groups of nodes; **(4)** inline `style` targets a single node. Each layer overrides the previous. The `base` theme is the best starting point for full custom branding.

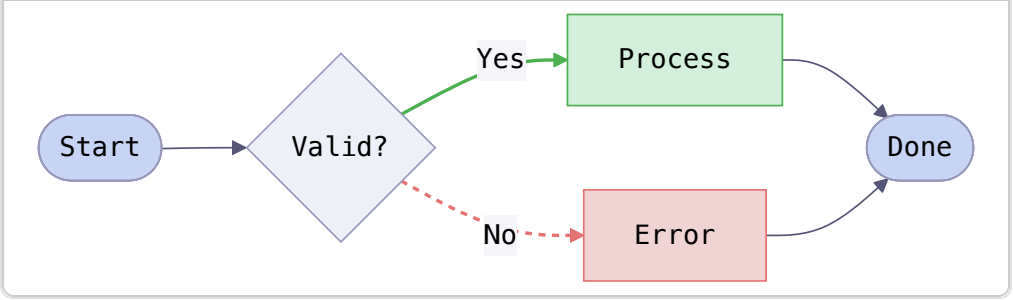
STYLING LAYERS

- `theme` — global palette preset
- `themeVariables` — token-level overrides
- `classDef` — reusable node classes
- `style id` — single-node inline style
- `linkStyle N` — per-edge line style
- External CSS — host-page class injection

CODE EXAMPLE — ALL STYLING TECHNIQUES

```
%%{init: {
  "theme": "base",
  "themeVariables": {
    "primaryColor":       "#2d2f52",
    "primaryTextColor":   "#e8eaf6",
    "primaryBorderColor": "#5c6bc0",
    "lineColor":          "#7986cb",
    "secondaryColor":     "#1c2030",
    "tertiaryColor":      "#252a3d",
    "background":         "#141720",
    "fontFamily":         "monospace"
  }
}}%%

flowchart LR
  %% ── classDef: reusable styles ──
  classDef success fill:#1a4d2e,stroke:#4caf50,color:#b
  classDef danger  fill:#4a1010,stroke:#e57373,color:#f
  classDef neutral fill:#252a3d,stroke:#5c6bc0,color:#c
  A([Start]) --> B{Valid?}
  B -- Yes --> C[Process]:::success
  B -- No  --> D[Error]:::danger
  C --> E([Done]):::neutral
  D --> E

  %% ── inline style overrides ──
  style A fill:#3a4a8c,color:#fff
  style E fill:#3a4a8c,color:#fff

  %% ── linkStyle by edge index ──
  linkStyle 1 stroke:#4caf50,stroke-w
  linkStyle 2 stroke:#e57373,stroke-w
```

RENDERED OUTPUT



BUILT-IN THEMES

THEME	CHARACTER
`default`	Light background, blue-grey palette — standard docs look
`dark`	Dark background, muted tones — terminals and dark UIs
`forest`	Light background, green accent palette
`neutral`	Greyscale — maximally neutral, good for print
`base`	Barebones; all colors set by `themeVariables` — best for custom branding

CLASSDEF & INLINE STYLE PROPERTIES

PROPERTY	CONTROLS
`fill`	Node background color
`color`	Node text color
`stroke`	Node border color
`stroke-width`	Border thickness, e.g. `2px`
`stroke-dasharray`	Dashed border, e.g. `5 3`
`font-size`	Node label font size
`font-weight`	Label weight: `bold` / `normal`
`rx / ry`	Border radius (SVG attributes)

KEY THEMEVARIABLES TOKENS

VARIABLE	AFFECTS
`primaryColor`	Main node fill
`primaryTextColor`	Text inside primary nodes
`primaryBorderColor`	Primary node border
`secondaryColor`	Secondary / subgraph fill
`tertiaryColor`	Third-level fill
`lineColor`	Edge / connector color
`background`	Diagram background
`fontFamily`	All diagram text
`fontSize`	Base font size (px string)
`edgeLabelBackground`	Edge label pill background

COMMON MISTAKES

- Using `style` instead of `classDef` when styling multiple nodes — results in repetition and drift
- `linkStyle` index counts all edges in declaration order from 0 — easy to mis-count in large diagrams
- Setting `themeVariables` on themes other than `base` — other themes partially ignore variable overrides
- Hex colors must include the `#` — bare hex values like `2d2f52` are silently ignored

TIPS & BEST PRACTICES

- Use `"theme": "base"` as your starting point for any branded or dark-mode diagram
- Define semantic `classDef` names: `success`, `danger`, `inactive` — not `green`, `red`
- Apply `:::className` inline on nodes to keep style declarations near usage
- For host-page CSS injection, target `.mermaid svg` — but inline styles take precedence

SUMMARY

- 4 styling layers: `theme` → `themeVariables` → `classDef` → `style id`
- Use `theme: base` + `themeVariables` for full custom control
- `classDef` names node groups; `class A,B name` or `A:::name` applies them
- `linkStyle N` styles edges by 0-based declaration index

Config & Integration

%%{init} options · themeVariables · HTML embed · CLI · security · platforms

CONFIGURATION OVERVIEW

Mermaid can be configured at three levels: **per-diagram** via `%%{init}%%` blocks at the top of the diagram source; **globally** via `mermaid.initialize(config)` when embedding in HTML; and **via CLI flags** when using `mmdc`. The `%%{init}%%` block always takes precedence for the diagram it's on. Configuration covers layout, security, fonts, diagram-specific options, and full `themeVariables` token overrides.

CONFIG PRIORITY

- `%%{init}%%` — highest: per-diagram override
- `mermaid.initialize()` — global page-level config
- CLI `--config` flag — file-level config for mmdc
- Mermaid defaults — lowest priority fallback
- Order: init block > initialize() > CLI > defaults

HTML EMBED — CDN

```
<!-- 1. Mark diagram containers -->
<pre class="mermaid">
  flowchart LR
    A --> B --> C
</pre>

<!-- 2. Load Mermaid from CDN -->
<script type="module">
  import mermaid from
    'https://cdn.jsdelivr.net/npm/mermaid@11/dist/mermaid.esm.min.mjs';

  mermaid.initialize({
    startOnLoad: true,
    theme: 'dark',
    securityLevel: 'loose',
    fontFamily: 'monospace',
    flowchart: {
      curve: 'basis',
      htmlLabels: true
    }
  });
</script>
```

CLI USAGE (MMDC)

```
# Install globally
npm install -g @mermaid-js/mermaid-cli

# Basic: .mmd → .svg
mmdc -i diagram.mmd -o output.svg

# Export as PNG at 2× scale
mmdc -i diagram.mmd -o output.png -s 2

# Apply external config JSON
mmdc -i diagram.mmd -o output.svg \
     --configFile config.json

# Apply custom CSS
mmdc -i diagram.mmd -o output.svg \
     --cssFile custom.css

# Dark theme + specific width
mmdc -i diagram.mmd -o output.png \
     -t dark -w 1200
```

%%{INIT} TOP-LEVEL OPTIONS

KEY	VALUES / NOTES
`theme`	`base` · `default` · `dark` · `forest` · `neutral`
`logLevel`	1=debug · 2=info · 3=warn · 4=error · 5=fatal
`securityLevel`	`strict` · `loose` · `antiscript` · `sandbox`
`startOnLoad`	`true` / `false` — auto-render `.mermaid` elements
`fontFamily`	CSS font string, e.g. `"trebuchet ms, sans-serif"`
`fontSize`	Number in px, e.g. 14
`htmlLabels`	`true` — enables HTML in node labels (flowchart)
`flowchart`	Sub-object: `curve` · `padding` · `useMaxWidth`
`sequence`	Sub-object: `diagramMarginX` · `actorMargin` · `boxMargin`
`gantt`	Sub-object: `barHeight` · `barGap` · `topPadding`

SECURITY LEVELS

LEVEL	BEHAVIOUR
`strict`	Default. HTML in labels escaped. Safest for public content.
`antiscript`	HTML allowed but scripts stripped.
`loose`	HTML + click events allowed. Only use on trusted content.
`sandbox`	Rendered in sandboxed iframe. Maximum isolation.

PLATFORM INTEGRATION QUICK REF

PLATFORM	NOTES
GitHub	Native in .md, .mdx, wikis — uses `strict` security
GitLab	Native in .md — same fenced block syntax
Obsidian	Built-in; config via plugin settings for theme
Docusaurus	`@docusaurus/theme-mermaid` — config in `docusaurus.config.js`
VitePress	Built-in markdown plugin — enable via `markdown.mermaid: true`
Hugo	Use shortcode or JS block — no native Mermaid support
Next.js / Astro	Install `mermaid` npm package, render client-side

COMMON MISTAKES

- Using `securityLevel: 'loose'` in a public-facing app — enables XSS if diagram source is user-controlled
- Loading Mermaid with a `<script src>` tag instead of ESM `import` — the CDN UMD build is deprecated in v10+
- Setting `startOnLoad: false` and forgetting to call `mermaid.run()` — diagrams never render
- CLI: forgetting `--scale` / `-s` for PNG exports — default 1× is often too low-res for docs

TIPS & BEST PRACTICES

- Pin your Mermaid CDN version (`mermaid@11.x.x`) to prevent breaking changes from minor updates
- Use `mermaid.run({ nodes: [...] })` to selectively render only specific elements on a page
- For SSG / static builds, use `mmdc` in a build step to pre-render diagrams as SVGs — no JS needed at runtime
- Store shared `%%{init}%%` config as a JS constant and inject it into diagrams programmatically to keep branding consistent

SUMMARY

- Config priority: `%%{init}%%` > `mermaid.initialize()` > CLI flags > defaults
- HTML embed: ESM `import` from CDN + `mermaid.initialize({ startOnLoad:true })`
- CLI: `mmdc -i in.mmd -o out.svg` ; add `-s 2` for hi-res PNG exports
- Security: use `strict` (default) for all public content; never use `loose` on user-supplied diagrams